

List Comprehension and Set Comprehension in Python

12 August 2026 11:14

Python provides a concise way to create collections using comprehensions.

The two important types are:

1. **List Comprehension** -> creates a list
2. **Set Comprehension** -> Creates a set

Instead of writing several lines of loops, we can often write the same logic in a single expression.

1. List Comprehension

A list comprehension is a compact way of creating a list from an iterable such as:

- list
- tuple
- range
- string
- set
- dictionary

General syntax:

`[expression for item in iterable]`

For example:

```
numbers = [1, 2, 3, 4, 5]
squares = [x * x for x in numbers]
print(numbers)
print(squares)
```

Output:

```
[1, 2, 3, 4, 5]
[1, 4, 9, 16, 25]
```

Traditional approach versus list comprehension

Suppose we want to create a list containing square of numbers from 1 to 5.

```
numbers = [1, 2, 3, 4, 5]
squares = []
for x in numbers:
```

```
squares.append(x * x)

print(numbers)
print(squares)
```

Output:

```
[1, 2, 3, 4, 5]
[1, 4, 9, 16, 25]
```

Using list comprehensions

```
numbers = [1, 2, 3, 4, 5]
squares = [x * x for x in numbers]
print(numbers)
print(squares)
```

The second version is shorter and more expressive.

List Comprehension with range()

```
numbers = [x for x in range(1, 11)]
squares = [x * x for x in numbers]
print(numbers)
print(squares)
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Generating even numbers and odd numbers:

```
even_num = [x for x in range(1, 11) if x % 2 == 0]
print(even_num)
odd_num = [x for x in range(1, 11) if x % 2 != 0]
print(odd_num)
```

Output:

```
[2, 4, 6, 8, 10]
[1, 3, 5, 7, 9]
```

List comprehension with strings.

A string is its arrival so we can use it with list comprehension.

```
name = "Python"
letters = [ch for ch in name]
print(letters)
```

Output:

```
['P', 'y', 't', 'h', 'o', 'n']
```

Converting characters to uppercase.

```
name = "Python"
letters = [ch.upper() for ch in name]
print(letters)
```

Output:

```
['P', 'Y', 'T', 'H', 'O', 'N']
```

Extracting vowels:

```
name = "Python Programming"
vowels = [ch for ch in name if ch in "AEIOUaeiou"]
print(vowels)
```

Output:

```
['o', 'o', 'a', 'i']
```

List comprehension with if - else

Syntax:

```
[expression_if_true if condition else expression_if_false]
```

```
numbers = [n for n in range(1, 6)]
result = ['even' if x % 2 == 0 else 'odd' for x in numbers]
print(numbers)
print(result)
```

Output:

```
[1, 2, 3, 4, 5]
['odd', 'even', 'odd', 'even', 'odd']
```

Finding Positive and Negative Values from a list

```
num = [-5, 10, 15, -20, 25, 30, -35]
positive = [x for x in num if x > 0]
negative = [x for x in num if x < 0]
print(positive)
print(negative)
```

Output:

```
[10, 15, 25, 30]
[-5, -20, -35]
```

Converting numbers to string

```
import random
num = [x*random.randint(1,10) for x in range(1, 11)]
num_str = [str(x) for x in num]
print(num)
print(num_str)
```

Output:

```
[3, 6, 15, 12, 30, 18, 70, 32, 90, 40]
['3', '6', '15', '12', '30', '18', '70', '32', '90', '40']
```

Nested List Comprehension

List comprehension can contain multiple for clauses.

```
[
    [1, 2, 3]
    [4, 5, 6]
    [7, 8, 9]
]
```

```
matrix = [
    [1,2,3],
    [4,5,6],
    [7,8,9]
]
result = [value for row in matrix for value in row ]
print(matrix)
print('Flattened the matrix')
```

```
print(result)
```

Output;

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]  
Flattened the matrix  
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Creating a multiplication table.

```
table = [[i*j for j in range(1, 11)] for i in range(1, 11)]  
for row in table:  
    print(row)
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]  
[3, 6, 9, 12, 15, 18, 21, 24, 27, 30]  
[4, 8, 12, 16, 20, 24, 28, 32, 36, 40]  
[5, 10, 15, 20, 25, 30, 35, 40, 45, 50]  
[6, 12, 18, 24, 30, 36, 42, 48, 54, 60]  
[7, 14, 21, 28, 35, 42, 49, 56, 63, 70]  
[8, 16, 24, 32, 40, 48, 56, 64, 72, 80]  
[9, 18, 27, 36, 45, 54, 63, 72, 81, 90]  
[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

List comprehension with a function

A function can also be used inside a comprehension.

```
def square(x):  
    return x * x  
num = [x for x in range(1,11)]  
res = [square(x) for x in num]  
print(num)  
print(res)
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

List Comprehension with user Input

```
num = [int(x) for x in input("Enter numbers: ").split()]
```

```
print(num)
```

Output:

```
Enter numbers: 10 20 30 40 50
[10, 20, 30, 40, 50]
```

Set Comprehension

A set comprehension is similar to list comprehension, but it creates a set.

Syntax:

```
{expression for item in itreable}
```

```
num = [int(x) for x in input("Enter numbers: ").split()]
print(num)
squares = { x * x for x in num}
print(squares)
```

Output:

```
Enter numbers: 1 2 3 4 5 6 7 8 9 10
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
{64, 1, 4, 36, 100, 9, 16, 49, 81, 25} ==> In Sets insertion order is not
maintained.
```

Why use set Comprehension?

Set automatically removes duplicate values.

We can do all the above concept using set just by applying curly braces '{}'

When Should You Use Comprehension?

Comprehensions are excellent when the operation is simple and readable. We should avoid making them excessively complicated.

In short: Use list comprehension when you want to construct a list and set comprehension when you want to construct a set - particularly when duplicate value should automatically be eliminated.